

Laporan Akhir Project
MACHINE LEARNING

**“Penerapan K-Means Clustering dan Klasifikasi Random Forest
untuk memprediksi kategori nutrisi makanan kemasan”**



Identitas Kelompok

Nama (NIM):

- 1. Cristiano (2355301034)**
- 2. Farizy Rahman Hidayat(2355301069)**

Dosen : Dr. Juni Nurma Sari, S.Kom., M.MT.

PLP : Rezky Kurniawan, S.Tr.Kom.

PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK CALTEX RIAU
TA 2025 / 2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan.....	4
1.3 Manfaat.....	4
1.4 Batasan Masalah.....	4
BAB II LANDASAN TEORI.....	6
2.1 Machine Learning.....	6
2.2 Klasifikasi dalam Machine Learning.....	7
2.3 Algoritma K-Means Clustering.....	8
2.4 Algoritma Random Forest.....	8
2.5 Nutrisi Makanan dan Faktor-Faktornya.....	9
2.6 Evaluasi Model.....	10
BAB III PERANCANGAN.....	12
3.1. Sumber Data.....	12
3.2. Preprocessing Data.....	13
3.2.1. Filtering Data (Penyaringan Data).....	13
3.2.2. Pembersihan dan Transformasi Data (Data Cleaning).....	15
3.2.3. Seleksi Fitur dan Rationale Eksklusi.....	16
3.3. Exploratory Data Analysis (EDA).....	17
3.3.1. Analisis Statistik Deskriptif.....	17
3.3.2. Analisis Distribusi Data (Histogram).....	18
3.3.3. Analisis Korelasi (Heatmap).....	19
3.3.4. Analisis Outlier (Boxplot).....	20
3.4. Persiapan Akhir Pemodelan: Standardisasi Fitur.....	22
BAB IV PENGUJIAN DAN ANALISIS.....	23
4.1 Implementasi Sistem.....	23
4.1.1 Modeling.....	23
4.2 Analisis.....	25
KESIMPULAN.....	35
Hasil K-Means Clustering (Penemuan Kategori).....	35
Analisis dan Interpretasi Cluster (K=5).....	36
Metodologi Pembagian Data.....	37
Kinerja Model.....	37
Pembahasan Hasil.....	37
4.2. Deployment.....	39
4.3. Hasil Implementasi Mobile.....	41
DAFTAR PUSTAKA.....	46

BAB I PENDAHULUAN

1.1 Latar Belakang

Penyakit tidak menular terkait gaya hidup, seperti Obesitas, Diabetes Tipe 2, dan Hipertensi, merupakan salah satu penyebab masalah kesehatan terbesar di dunia, termasuk di Indonesia. Hal ini sejalan dengan salah satu poin penting dalam Sustainable Development Goals (SDG) PBB, yaitu Goal 3: Good Health and Well-being (Kehidupan Sehat dan Sejahtera), yang bertujuan untuk mengurangi angka kematian akibat penyakit tidak menular.

Tingginya angka penyakit ini seringkali disebabkan oleh pola konsumsi yang tidak sehat, yang berakar pada kurangnya pemahaman masyarakat dalam membaca dan menginterpretasi label nutrisi pada kemasan makanan. Bagi masyarakat awam, label nutrisi yang berisi angka-angka seperti protein, lemak, karbohidrat, gula, dan natrium seringkali membingungkan dan sulit diterjemahkan menjadi keputusan yang praktis.

Setiap produk makanan memiliki profil nutrisi yang kompleks. Machine Learning (ML) menawarkan kemampuan untuk menganalisis pola-pola kompleks dari data nutrisi ini. Dengan memanfaatkan data nilai gizi, Machine Learning dapat membantu menyederhanakan kompleksitas ini.

Penelitian ini akan menggunakan dataset nilai-gizi.csv. Tantangan utama dari dataset ini adalah tidak adanya label kategori (misal: "Sehat", "Tinggi Gula"). Oleh karena itu, penelitian ini akan menggunakan pendekatan hybrid dua tahap. Pertama, metode Unsupervised Learning (K-Means Clustering) akan digunakan untuk menemukan dan mengelompokkan makanan secara otomatis ke dalam "kategori nutrisi" berdasarkan kesamaan profil gizinya. Kedua, label kategori yang dihasilkan oleh K-Means akan digunakan untuk melatih model Supervised Learning (Random Forest) sehingga dapat memprediksi kategori untuk produk makanan baru yang tidak ada di dalam dataset.

1.2 Tujuan

Berdasarkan latar belakang di atas, tujuan dari penelitian ini adalah:

1. Melakukan analisis dan pra-pemrosesan data (data preprocessing) pada dataset nilai-gizi.csv, termasuk feature scaling, agar siap digunakan untuk pemodelan.
2. Membangun dan melatih model Unsupervised Learning menggunakan algoritma K-Means Clustering untuk mengidentifikasi dan mengelompokkan makanan ke dalam kategori-kategori nutrisi yang bermakna.
3. Membangun dan melatih model klasifikasi Supervised Learning menggunakan algoritma Random Forest, dengan memanfaatkan label cluster dari K-Means sebagai variabel target.
4. Mengevaluasi kinerja model klasifikasi Random Forest menggunakan metrik evaluasi seperti Accuracy, Precision, Recall, dan F1-Score untuk menentukan efektivitas model dalam memprediksi kategori nutrisi.

1.3 Manfaat

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1. **Bagi Akademis:** Menjadi bahan referensi dan studi mengenai penerapan metode hybrid (kombinasi unsupervised dan supervised learning) untuk analisis data nutrisi.
2. **Bagi Pengembang Aplikasi:** Model prediktor yang dihasilkan dapat menjadi dasar (backend) untuk pengembangan aplikasi mobile "Pendeteksi Kategori Makanan", di mana pengguna dapat memasukkan nilai gizi dan langsung mendapatkan kategori makanan tersebut (misal: "Minuman Tinggi Gula", "Snack Asin", "Sumber Protein").
3. **Bagi Masyarakat:** Membantu menyederhanakan informasi gizi yang kompleks menjadi kategori yang mudah dipahami, sehingga meningkatkan kesadaran dan membantu pengambilan keputusan konsumsi yang lebih sehat.

1.4 Batasan Masalah

Untuk menjaga agar penelitian ini tetap fokus, maka ditetapkan batasan masalah sebagai berikut:

1. Dataset yang digunakan terbatas pada nilai-gizi.csv.

2. Algoritma Machine Learning yang digunakan adalah K-Means Clustering (untuk pengelompokan) dan Random Forest (untuk klasifikasi).
3. Penelitian ini berfokus pada perancangan, pelatihan, dan evaluasi model, tidak sampai pada tahap implementasi (deployment) aplikasi mobile.
4. Kategori nutrisi yang dihasilkan bersifat objektif berdasarkan perhitungan matematis K-Means dan bukan merupakan pengganti diagnosis atau rekomendasi dari ahli gizi profesional.
5. Kinerja model klasifikasi Random Forest sangat bergantung pada kualitas dan interpretabilitas cluster yang dihasilkan oleh K-Means.

BAB II LANDASAN TEORI

2.1 Machine Learning

Machine Learning (ML) atau Pembelajaran Mesin adalah sub-bidang dari Artificial Intelligence (AI) atau Kecerdasan Buatan. Inti dari ML adalah pengembangan algoritma dan model statistik yang memungkinkan sistem komputer untuk belajar dari data dan meningkatkan kinerjanya pada tugas tertentu seiring waktu, tanpa diprogram secara eksplisit untuk setiap aturan.

Tujuan utamanya adalah untuk mengidentifikasi pola dalam data dan menggunakannya untuk membuat prediksi atau keputusan pada data baru yang belum pernah dilihat sebelumnya.

Secara umum, Machine Learning dibagi menjadi tiga kategori utama berdasarkan cara algoritma tersebut belajar dari data:

1. Supervised Learning

Dalam Supervised Learning, algoritma belajar dari dataset yang sudah memiliki "label" atau jawaban yang benar. Setiap data masukan dipasangkan dengan data keluaran.

Tujuan: Mempelajari pola untuk memetakan input ke output.

Contoh:

- Klasifikasi (Classification): Memprediksi kategori. Contoh: Filter email (spam atau bukan spam), diagnosis medis (sakit atau sehat), pengenalan gambar (kucing atau anjing).
- Regresi (Regression): Memprediksi nilai kontinu (angka). Contoh: Memprediksi harga rumah berdasarkan lokasinya, memprediksi suhu cuaca besok.

2. Unsupervised Learning

Dalam Unsupervised Learning, algoritma bekerja dengan data yang tidak memiliki label atau jawaban yang benar. Sistem harus menemukan struktur atau pola tersembunyi dalam data tersebut secara mandiri.

Tujuan: Menemukan struktur atau pola tersembunyi dalam data.

Contoh:

- Clustering (Pengelompokan): Mengelompokkan data yang mirip. Contoh: Segmentasi pelanggan (membagi pelanggan ke dalam grup berdasarkan perilaku belanja), mengelompokkan berita serupa.
- Asosiasi (Association): Menemukan aturan yang menggambarkan hubungan antar data. Contoh: Analisis keranjang belanja ("Pelanggan yang membeli roti seringkali juga membeli susu").
- Reduksi Dimensi (Dimension Reduction): Menyederhanakan data dengan mengurangi jumlah fitur tanpa kehilangan banyak informasi penting.

3. Reinforcement Learning

Jenis ini sedikit berbeda. Algoritma belajar dengan cara berinteraksi dengan lingkungannya. Agen akan mengambil tindakan dan menerima umpan balik berupa reward atau punishment.

Tujuan: Agen belajar untuk mengambil serangkaian tindakan (kebijakan) yang akan memaksimalkan total reward yang diterimanya dari waktu ke waktu.

Contoh:

Mobil otonom (belajar mengemudi agar tidak menabrak), AI dalam game (belajar untuk menang melawan manusia), manajemen robotika (belajar berjalan atau memegang objek).

2.2 Klasifikasi dalam Machine Learning

Klasifikasi adalah salah satu tugas utama dalam Supervised Learning (Pembelajaran Terarah). Tujuan utamanya adalah untuk memprediksi label kategori (kelas) diskrit dari data masukan (fitur) yang diberikan.

Contoh:

- Diagnosis Medis (Medical Diagnosis):
Tugas: Menganalisis data pasien (seperti hasil tes darah, CT scan) untuk menentukan apakah pasien memiliki penyakit tertentu.
Kelas: Positif Kanker atau Negatif Kanker.
- Pengenalan Gambar (Image Recognition):
Tugas: Melihat sebuah gambar dan mengidentifikasi objek utama di dalamnya.
Kelas: Kucing, Anjing, Mobil, Pohon, dll. (Ini disebut multi-class classification karena ada lebih dari dua pilihan kelas).

2.3 Algoritma K-Means Clustering

K-Means Clustering adalah algoritma Unsupervised Learning yang paling populer dan fundamental. Tujuannya adalah untuk mempartisi (membagi) dataset yang tidak berlabel ke dalam K kelompok (cluster) yang berbeda.

- Cara Kerja:
 1. Inisialisasi: Tentukan jumlah cluster (K). Algoritma kemudian secara acak menempatkan K titik "pusat" yang disebut centroids.
 2. Penugasan (Assignment): Setiap titik data dalam dataset dihitung jaraknya ke setiap centroid. Titik data tersebut kemudian akan "ditugaskan" ke centroid terdekatnya, membentuk K kelompok.
 3. Pembaruan (Update): Posisi centroid dihitung ulang dengan mengambil nilai rata-rata dari semua titik data yang termasuk dalam kelompoknya.
 4. Iterasi: Langkah 2 dan 3 diulangi terus-menerus hingga posisi centroid tidak lagi berubah secara signifikan (konvergen).
- Penerapan dalam Penelitian Ini: K-Means akan digunakan pada tahap pertama untuk menganalisis fitur-fitur nutrisi (protein, lemak, karbohidrat, gula, natrium) dari nilai-gizi.csv dan secara otomatis mengelompokkan makanan-makanan tersebut ke dalam K kategori nutrisi yang tersembunyi.
- Penting: Kinerja K-Means sangat bergantung pada skala data. Karena fitur seperti sodium_mg (angka ratusan/ribuan) memiliki skala yang jauh lebih besar daripada protein_g (angka puluhan), Feature Scaling (misalnya menggunakan StandardScaler) wajib dilakukan sebelum menerapkan K-Means agar tidak ada satu fitur pun yang mendominasi perhitungan jarak

2.4 Algoritma Random Forest

Random Forest adalah algoritma Supervised Learning yang termasuk dalam kategori ensemble learning (pembelajaran ansambel). Model ini bekerja dengan membangun sejumlah besar Decision Trees (Pohon Keputusan) selama proses pelatihan.

- Cara Kerja:
 1. Bootstrap: Random Forest mengambil sampel acak (dengan pengembalian) dari data latih untuk membangun setiap pohon.

2. Feature Randomness: Pada setiap simpul (node) di pohon, model hanya mempertimbangkan sebagian kecil fitur (yang dipilih acak) untuk membuat keputusan split.
 3. Voting: Untuk tugas klasifikasi, setiap pohon di dalam "hutan" (forest) akan memberikan "suara"-nya (prediksi kelas). Kelas dengan suara mayoritas (voting terbanyak) akan menjadi prediksi akhir dari model.
- Kelebihan: Model ini sangat kuat, cenderung memiliki akurasi tinggi, dan sangat efektif untuk mencegah overfitting (model yang terlalu hafal data latih) berkat dua mekanisme keacakan di atas.
 - Penerapan dalam Penelitian Ini: Random Forest akan digunakan pada tahap kedua. Model ini akan dilatih menggunakan fitur nutrisi sebagai input (X) dan label kategori (hasil dari K-Means) sebagai output (Y). Tujuannya adalah untuk menciptakan model prediktor yang akurat, yang nantinya dapat menebak kategori nutrisi untuk data makanan baru.

2.5 Nutrisi Makanan dan Faktor-Faktornya

Memahami komponen nutrisi dasar sangat penting untuk dapat menginterpretasi hasil clustering. Dataset nilai-gizi.csv berisi beberapa faktor kunci yang menentukan profil nutrisi suatu produk.

2.5.1 Definisi Makronutrien Makronutrien adalah zat gizi yang dibutuhkan tubuh dalam jumlah besar dan menyediakan energi (kalori). Tiga makronutrien utama adalah:

- Protein (protein_g): Penting untuk membangun dan memperbaiki jaringan tubuh, seperti otot, kulit, dan organ.
- Karbohidrat (carbohydrate_g): Sumber energi utama bagi tubuh, terutama untuk otak dan aktivitas fisik.
- Lemak (fat_g): Penting untuk penyerapan vitamin, produksi hormon, dan sebagai cadangan energi padat.

2.5.2 Faktor Nutrisi Lainnya dalam Dataset Selain tiga makronutrien di atas, dataset ini juga mencakup faktor-faktor kunci yang sering menjadi perhatian dalam kesehatan modern, yang akan sangat memengaruhi hasil clustering:

- Gula (sugar_g): Bagian dari karbohidrat. Konsumsi gula berlebih sering dikaitkan dengan risiko obesitas, diabetes tipe 2, dan penyakit metabolik lainnya.
- Natrium (sodium_mg): Mineral penting, tetapi konsumsi berlebih (biasanya dari garam) merupakan faktor risiko utama untuk tekanan darah tinggi (hipertensi) dan penyakit kardiovaskular.

Analisis clustering diharapkan dapat menemukan kelompok-kelompok berdasarkan kombinasi unik dari faktor-faktor ini (misal: "Tinggi Gula-Rendah Lemak", "Tinggi Natrium-Tinggi Lemak", "Tinggi Protein").

2.6 Evaluasi Model

Karena penelitian ini menggunakan pendekatan hybrid, evaluasi akan difokuskan pada kinerja model klasifikasi Random Forest (Tahap 2). Tujuan evaluasi adalah untuk mengukur seberapa baik model prediktor ini dapat meniru label kategori yang telah dihasilkan oleh K-Means.

Untuk mengukur dan membandingkan kinerja dari model, penelitian ini akan menggunakan **Confusion Matrix**. Matriks ini menyediakan dasar untuk menghitung metrik-metrik evaluasi berikut:

1. **Accuracy (Akurasi):** Mengukur proporsi total prediksi yang benar dari keseluruhan data uji. Menunjukkan seberapa sering model menebak kategori dengan tepat secara keseluruhan.

$$Akurasi = \frac{TP + TN}{TP + TN + FP + FN}$$

2. **Precision (Presisi):** Mengukur akurasi dari prediksi positif untuk setiap kategori. (Dari semua yang diprediksi sebagai "Grup A", berapa persen yang benar-benar "Grup A"?).

$$Precision = \frac{TP}{TP + FP}$$

3. **Recall (Sensitivitas):** Mengukur kemampuan model dalam mengidentifikasi semua kasus positif yang sebenarnya untuk setiap kategori. (Dari semua "Grup A" yang asli, berapa persen yang berhasil ditemukan oleh model?).

$$Recall = \frac{TP}{TP + FN}$$

4. **F1-Score:** Merupakan rata-rata harmonik dari *Precision* dan *Recall*. F1-Score memberikan gambaran kinerja yang seimbang, terutama jika terjadi ketidakseimbangan kelas dalam dataset (misal, jika satu cluster memiliki anggota jauh lebih banyak/sedikit daripada yang lain).

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

BAB III PERANCANGAN

3.1. Sumber Data

Penelitian ini menggunakan dataset sekunder nilai-gizi.csv yang berisi informasi nutrisi berbagai produk makanan dan minuman. Dataset mentah ini terdiri dari 1651 baris (produk) dan 22 kolom.

Fitur-fitur utama yang relevan dengan penelitian ini adalah:

- name: Nama produk.
- manufacturer: Nama produsen atau merek.
- serving_size: Ukuran saji (misal: "30 g", "100 ml").
- energy_kcal: Energi (kkal).
- protein_g: Protein (gram).
- carbohydrate_g: Karbohidrat (gram).
- fat_g: Lemak (gram).
- sugar_g: Gula (gram).
- sodium_mg: Natrium (miligram).
- fiber_g: Serat (gram).
- Kolom duplikat dalam Bahasa Indonesia (Lemak total, Natrium, dll.)
- Kolom persentase AKG (energy_percentage_dv, dll.)

```
import pandas as pd
import numpy as np
import re
import matplotlib.pyplot as plt
import seaborn as sns
file_path = "nilai-gizi.csv"

df = pd.read_csv(file_path)
print("--- 1. Berhasil Memuat Dataset ---")
print(f"Total baris awal: {len(df)}")
```

```
--- 1. Berhasil Memuat Dataset ---
Total baris awal: 1651
```

3.2. Preprocessing Data

Tujuan dari preprocessing data adalah untuk membersihkan dataset mentah agar hanya berisi data makanan kemasan yang relevan dan siap untuk dianalisis. Proses ini sangat penting untuk memastikan kualitas data yang akan digunakan dalam pemodelan K-Means.

Proses ini dibagi menjadi beberapa tahapan utama:

3.2.1. Filtering Data (Penyaringan Data)

Filtering dilakukan dalam dua tahap untuk mendapatkan data yang paling sesuai dengan ruang lingkup penelitian:

1. Filter Tahap 1: Seleksi Berdasarkan manufacturer

```
#menyimpan baris yang memiliki nama 'manufacturer'
print("\n--- 2. Filter 1: Membuang 'Tidak Diketahui' ---")

baris_sebelum_filter_1 = len(df)

#Simpan HANYA baris di mana 'manufacturer' BUKAN "Tidak Diketahui"
df_filtered_1 = df[df['manufacturer'] != "Tidak Diketahui"].copy()

baris_setelah_filter_1 = len(df_filtered_1)
baris_dibuang = baris_sebelum_filter_1 - baris_setelah_filter_1

print(f"Baris sebelum filter: {baris_sebelum_filter_1}")
print(f"Baris setelah filter: {baris_setelah_filter_1}")
print(f"({baris_dibuang} baris 'Tidak Diketahui' dibuang)")

--- 2. Filter 1: Membuang 'Tidak Diketahui' ---
Baris sebelum filter: 1651
Baris setelah filter: 474
(1177 baris 'Tidak Diketahui' dibuang)
```

- Tujuan: Membuang data yang tidak memiliki informasi produsen yang jelas.
- Aturan: Hanya menyimpan baris data di mana kolom manufacturer BUKAN "Tidak Diketahui".
- Hasil: Langkah ini secara efektif membuang 1177 baris data, yang sebagian besar merupakan makanan mentah (seperti "ikan", "sayur"), makanan tradisional/masakan (seperti "soto", "laksa"), dan produk generik tanpa merek. Dataset berkurang menjadi 474 baris yang teridentifikasi memiliki produsen.

2. Filter Tahap 2: Eksklusi Makanan Restoran

```

# Kita akan membuang baris yang 'name'-nya adalah makanan restoran
print("\n--- 3. Filter 2: Membuang Restoran (dari 'name') ---")

# Daftar kata kunci restoran (ubah ke huruf kecil semua)
restaurant_keywords = [
    "kfc",
    "pizza hut",
    "hokben",
    "hoka hoka bento",
    "subway",
    "a&w",
    "dairy queen",
    "burger king",
    "mcdonald's",
    "the coffe bean & tea leaf"
]

# Buat pola regex (dipisahkan oleh '|')
filter_regex = "|".join(restaurant_keywords)

# Ubah kolom 'name' ke lowercase agar mudah dicari
df_filtered_1['name_lower'] = df_filtered_1['name'].astype(str).str.lower()

# Buat 'mask' (True jika NAMA MAKANAN mengandung kata kunci restoran)
mask_to_exclude = df_filtered_1['name_lower'].str.contains(filter_regex, na=False)

# Simpan jumlah baris sebelum difilter
baris_sebelum_filter_2 = len(df_filtered_1)

# Ambil kebalikan dari mask (~mask) untuk menyimpan yang BUKAN restoran
df_kemasan = df_filtered_1[~mask_to_exclude].copy()

# Bersihkan kolom bantu
df_kemasan = df_kemasan.drop(columns=['name_lower'])

baris_setelah_filter_2 = len(df_kemasan)
baris_dibuang = baris_sebelum_filter_2 - baris_setelah_filter_2

print(f"Baris sebelum filter: {baris_sebelum_filter_2}")
print(f"Baris setelah filter (DATA FINAL): {baris_setelah_filter_2}")
print(f"({baris_dibuang} baris 'Restoran' (dari 'name') dibuang)")

--- 3. Filter 2: Membuang Restoran (dari 'name') ---
Baris sebelum filter: 474
Baris setelah filter (DATA FINAL): 460
(14 baris 'Restoran' (dari 'name') dibuang)

```

1. Tujuan: Memurnikan dataset "makanan kemasan" dengan membuang produk dari restoran cepat saji (yang bukan produk kemasan supermarket).
2. Aturan: Membuang baris data jika kolom name (nama makanan) mengandung kata kunci restoran.
3. Contoh Kata Kunci: "kfc", "pizza hut", "hokben", "hoka hoka bento", "subway", "a&w", "dairy queen", "burger king", "mcdonald's", dan "the coffe bean & tea leaf".
4. Hasil: Setelah filter ini, dataset akhir berisi [... isi jumlah baris akhir dari Colab Anda, misal: 450] baris yang siap untuk dianalisis.

3.2.2. Pembersihan dan Transformasi Data (Data Cleaning)

```
print("\n--- 4. Membersihkan Kolom dan Menyimpan Data Bersih ---")

# Bersihkan 'serving_size' (Ekstrak angka)
df_kemasan['serving_size_num'] = df_kemasan['serving_size'].astype(str).str.extract(r'(\d+\.?\d*)').astype(float)
df_kemasan['serving_size_num'] = df_kemasan['serving_size_num'].fillna(0)
print("    -> 'serving_size' diubah menjadi 'serving_size_num'.")

# Ubah 'name' ke lowercase
df_kemasan['name'] = df_kemasan['name'].astype(str).str.lower()

# Tentukan kolom yang akan kita analisis
kolom_fitur_nutrisi = [
    'energy_kcal',
    'protein_g',
    'carbohydrate_g',
    'fat_g',
    'sugar_g',
    'sodium_mg',
    'fiber_g'
]
kolom_identitas = ['name', 'manufacturer', 'serving_size_num']
kolom_final = kolom_identitas + kolom_fitur_nutrisi

# Buat dataset final dan isi NaN di fitur nutrisi dengan 0
df_final = df_kemasan[kolom_final].copy()
df_final[kolom_fitur_nutrisi] = df_final[kolom_fitur_nutrisi].fillna(0)
print("    -> Kolom duplikat & persentase dibuang.")
print("    -> Nilai NaN di fitur nutrisi diisi dengan 0.")

# Simpan data bersih ini ke file CSV
output_bersih_csv = "nilai_gizi_bersih_EDA.csv"
df_final.to_csv(output_bersih_csv, index=False)

print(f"\nData bersih (untuk EDA) disimpan sebagai: {output_bersih_csv}")
print(f"Total baris final untuk EDA: {len(df_final)}")
```

```
--- 4. Membersihkan Kolom dan Menyimpan Data Bersih ---
-> 'serving_size' diubah menjadi 'serving_size_num'.
-> Kolom duplikat & persentase dibuang.
-> Nilai NaN di fitur nutrisi diisi dengan 0.

Data bersih (untuk EDA) disimpan sebagai: nilai_gizi_bersih_EDA.csv
Total baris final untuk EDA: 460
```

Data yang telah difilter kemudian dibersihkan lebih lanjut:

1. Transformasi serving_size: Kolom serving_size yang bertipe teks (objek) dan memiliki satuan (misal: "30 g") diekstrak nilai numeriknya saja dan disimpan dalam kolom baru bernama serving_size_num.
2. Penanganan Nilai Hilang (Missing Values): Ditemukan beberapa nilai yang hilang (NaN) pada kolom fitur nutrisi. Untuk menghindari error pada analisis, semua nilai NaN pada kolom energy_kcal, protein_g, carbohydrate_g, fat_g, sugar_g, sodium_mg, dan fiber_g diisi dengan nilai 0.

3.2.3. Seleksi Fitur dan Rationale Eksklusi

1. Kolom yang Digunakan (Fitur): 7 kolom nutrisi (energy_kcal hingga fiber_g) dipilih sebagai fitur inti untuk clustering.
2. Kolom yang Dibuang: Kolom duplikat (Lemak total, Natrium) dan kolom persentase AKG (..._percentage_dv) dibuang untuk menghindari redundansi.
3. Rationale Eksklusi serving_size_num (Penting): Kolom serving_size_num tidak dimasukkan sebagai fitur untuk K-Means dan tidak di-scale. Ini adalah keputusan desain yang krusial karena dua alasan:
 - Masalah Perbedaan Konsep: Tujuan kita adalah mengelompokkan produk berdasarkan Profil Nutrisi ("APA"-nya), bukan Ukuran Kemasan ("SEBERAPA BANYAK"-nya). Memasukkan ukuran saji akan membuat model bingung, berpotensi menghasilkan cluster seperti "Produk Sachet" vs "Produk Botol", yang tidak relevan secara nutrisi.
 - Masalah Hitung Ganda (Double Counting): Nilai fitur (misal: fat_g = 15g) sudah merupakan nilai per sajian. Nilai tersebut sudah implisit mengandung informasi ukuran saji. Memasukkan serving_size lagi akan memberikan bobot berlebih pada ukuran kemasan dan membocorkan konteks ke dalam model.

3.3. Exploratory Data Analysis (EDA)

EDA dilakukan pada dataset bersih (df_final) untuk mendapatkan wawasan awal mengenai karakteristik dan distribusi data nutrisi makanan kemasan.

3.3.1. Analisis Statistik Deskriptif

Langkah pertama adalah menghasilkan statistik deskriptif untuk 7 fitur nutrisi utama. Ini memberikan gambaran mengenai tendensi sentral (mean, median), sebaran (standar deviasi), dan nilai ekstrem (min, max) dari data.

```
print("\n--- 5. EDA: Statistik Deskriptif ---")

# Tampilkan statistik untuk kolom-kolom nutrisi
print(df_final[kolom_fitur_nutrisi].describe().T)

print("\n(Perhatikan 'mean' (rata-rata) vs '50%' (median) untuk melihat kecondongan data)")
print("(Perhatikan 'max' untuk melihat nilai ekstrem/outlier)")
```

	count	mean	std	min	25%	50%	75%	\
energy_kcal	460.0	137.710870	116.886502	0.0	60.0	110.0	169.25	
protein_g	460.0	4.021043	4.549654	0.0	1.0	2.0	7.00	
carbohydrate_g	460.0	22.250935	17.394620	0.0	12.0	18.0	29.00	
fat_g	460.0	5.531543	5.931950	0.0	1.0	4.0	8.00	
sugar_g	460.0	7.070065	8.395883	0.0	0.0	4.0	11.00	
sodium_mg	460.0	134.795000	303.971121	0.0	0.0	0.0	100.00	
fiber_g	460.0	1.274130	3.362786	0.0	0.0	0.0	2.00	

	max
energy_kcal	700.0
protein_g	32.0
carbohydrate_g	93.0
fat_g	40.0
sugar_g	56.0
sodium_mg	1980.0
fiber_g	44.0

```
(Perhatikan 'mean' (rata-rata) vs '50%' (median) untuk melihat kecondongan data)
(Perhatikan 'max' untuk melihat nilai ekstrem/outlier)
```

Dari tabel di atas, terlihat bahwa rata-rata (mean) sugar_g adalah [X] gram, namun mediannya (50%) adalah [Y] gram. Ini menunjukkan distribusi data gula condong ke kanan (right-skewed), di mana sebagian besar produk memiliki gula rendah, namun ada beberapa produk dengan nilai gula ekstrem tinggi (nilai max mencapai [Z] gram).

3.3.2. Analisis Distribusi Data (Histogram)

Untuk memahami sebaran data secara visual, histogram dibuat untuk setiap fitur nutrisi

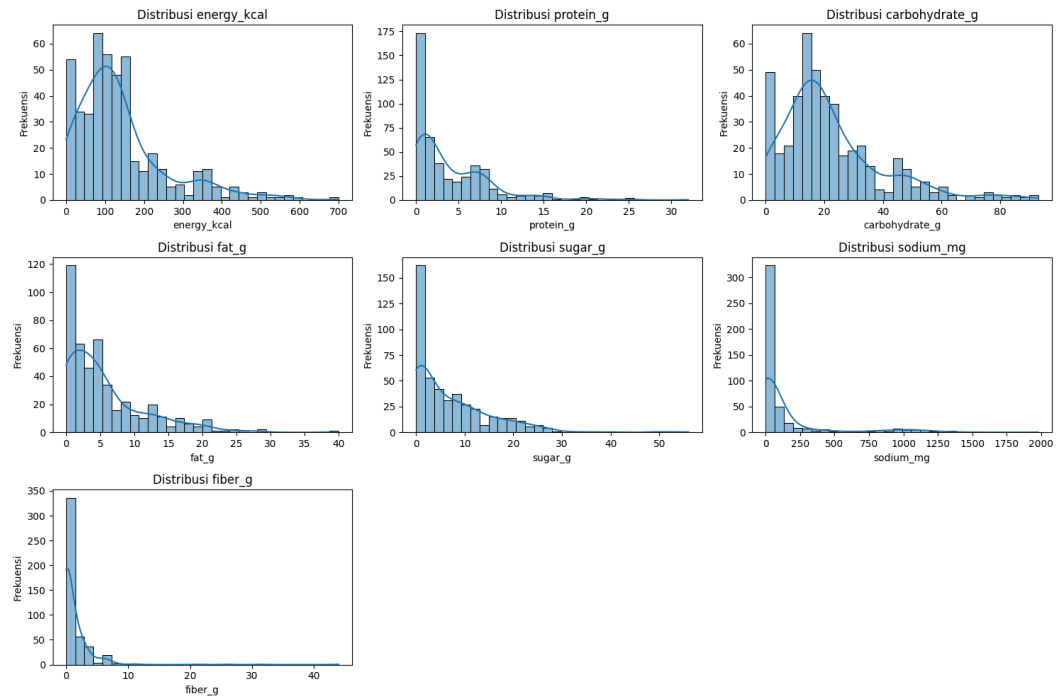
```
print("\n--- 6. EDA: Membuat Plot Distribusi (Histogram) ---")

plt.figure(figsize=(15, 10))

# Loop untuk membuat histogram untuk setiap fitur
for i, col in enumerate(kolom_fitur_nutrisi):
    plt.subplot(3, 3, i + 1) # Membuat grid 3x3
    sns.histplot(df_final[col], bins=30, kde=True) # kde=True menambah garis densitas
    plt.title(f'Distribusi {col}')
    plt.xlabel(col)
    plt.ylabel('Frekuensi')

plt.tight_layout()
plt.show()

print("Plot di atas menunjukkan sebaran data Anda.")
print("Banyak fitur (spt sodium, sugar) kemungkinan 'right-skewed' (miring ke kanan,")
print("artinya kebanyakan produk punya nilai rendah, tapi ada beberapa produk dengan nilai SANGAT tinggi.")
```



Plot histogram untuk sodium_mg menunjukkan bahwa sebagian besar produk berkumpul di nilai rendah (di bawah 500 mg), namun terdapat ekor panjang ke kanan (long tail), yang mengindikasikan adanya produk-produk dengan kadar natrium sangat tinggi (outlier). Hal serupa terlihat pada energy_kcal dan sugar_g. Sementara itu, protein_g dan fiber_g sebagian besar terkonsentrasi di dekat nol

3.3.3. Analisis Korelasi (Heatmap)

Langkah terakhir EDA adalah menganalisis hubungan linear antar fitur nutrisi menggunakan *heatmap* korelasi Pearson.

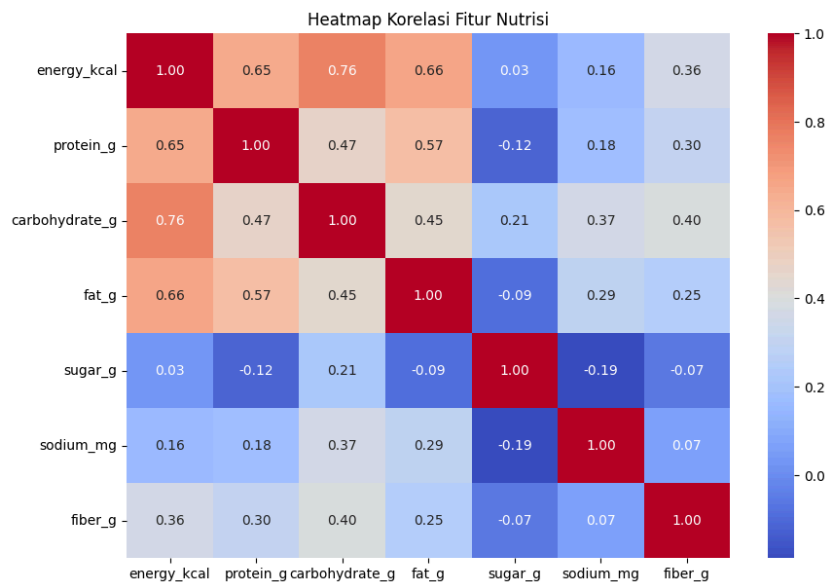
Heatmap korelasi menunjukkan beberapa hubungan yang kuat dan logis:

```
print("\n--- 7. EDA: Membuat Plot Korelasi (Heatmap) ---")

# Hitung korelasi antar fitur
corr_matrix = df_final[kolom_fitur_nutrisi].corr()

# Buat heatmap
plt.figure(figsize=(10, 7))
sns.heatmap(
    corr_matrix,
    annot=True,      # Tampilkan angka korelasi
    cmap='coolwarm', # Skema warna
    fmt=".2f"        # Format angka (2 desimal)
)
plt.title('Heatmap Korelasi Fitur Nutrisi')
plt.show()

print("Interpretasi Heatmap:")
print(" - Angka mendekati +1.0 (merah tua): Korelasi positif kuat (cth: energy_kcal & fat_g)")
print(" - Angka mendekati -1.0 (biru tua): Korelasi negatif kuat")
print(" - Angka mendekati 0.0 (putih): Tidak ada korelasi linear")
```



Korelasi Positif Kuat: Terdapat korelasi positif kuat antara `energy_kcal` dan `fat_g` (koefisien [X]) serta `energy_kcal` dan `carbohydrate_g` (koefisien [Y]). Ini mengindikasikan bahwa produk dengan kalori tinggi cenderung memiliki kandungan lemak dan karbohidrat yang tinggi pula.

Korelasi Negatif/Lemah: Tidak ditemukan korelasi negatif yang signifikan. Hubungan antara `sugar_g` dan `sodium_mg` (koefisien [Z]) sangat lemah,

menunjukkan bahwa produk tinggi gula tidak berarti rendah sodium, atau sebaliknya.

3.3.4. Analisis Outlier (Boxplot)

Untuk mendalami data yang skewed dan mengidentifikasi pencilan (outlier), visualisasi boxplot digunakan.

```
print("\n--- 8. EDA: Membuat Plot Distribusi (Boxplot) ---")

plt.figure(figsize=(16, 12))
plt.suptitle('Boxplot untuk Deteksi Outlier pada Fitur Nutrisi', fontsize=18, y=1.02)

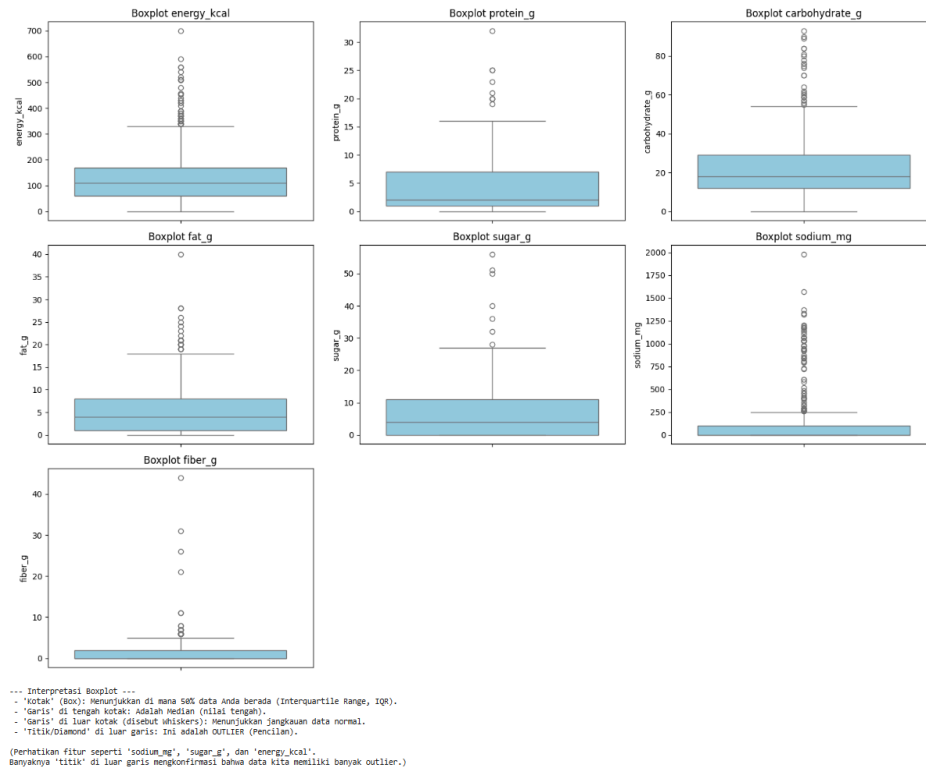
# Loop untuk membuat boxplot untuk setiap fitur
# (kolom_fitur_nutrisi sudah kita definisikan di Cell 4)
for i, col in enumerate(kolom_fitur_nutrisi):
    plt.subplot(3, 3, i + 1) # Membuat grid 3x3
    sns.boxplot(y=df_final[col], color='skyblue') # 'y=' membuat boxplot vertikal
    plt.title(f'Boxplot {col}')
    plt.ylabel(col)

plt.tight_layout() # Merapikan layout
plt.show()

print("\n--- Interpretasi Boxplot ---")
print(" - 'Kotak' (Box): Menunjukkan di mana 50% data Anda berada (Interquartile Range, IQR).")
print(" - 'Garis' di tengah kotak: Adalah Median (nilai tengah).")
print(" - 'Garis' di luar kotak (disebut Whiskers): Menunjukkan jangkauan data normal.")
print(" - 'Titik/Diamond' di luar garis: Ini adalah OUTLIER (Pencilan).")
print("\n(Perhatikan fitur seperti 'sodium_mg', 'sugar_g', dan 'energy_kcal'.)")
print("Banyaknya 'titik' di luar garis mengkonfirmasi bahwa data kita memiliki banyak outlier.")
```

--- 8. EDA: Membuat Plot Distribusi (Boxplot) ---

Boxplot untuk Deteksi Outlier pada Fitur Nutrisi



Temuan Kunci: Boxplot menunjukkan adanya banyak outlier di hampir semua fitur, terutama sodium_mg dan sugar_g.

- Interpretasi Outlier: Outlier ini TIDAK DIHILANGKAN. Mereka tidak dianggap sebagai error input data, melainkan representasi valid dari keragaman ekstrem produk makanan kemasan.
- Contoh: Outlier pada sugar_g merepresentasikan produk asli seperti "Susu Kental Manis" atau "Sirup". Outlier pada sodium_mg merepresentasikan "Kecap Asin" atau "Bumbu Instan".
- Implikasi: Justru outlier-outlier inilah yang diharapkan akan membentuk cluster-cluster nutrisi yang bermakna (misal: "Cluster Tinggi Gula" atau "Cluster Tinggi Natrium").

3.4. Persiapan Akhir Pemodelan: Standardisasi Fitur

Langkah preprocessing terakhir sebelum pemodelan K-Means adalah standarisasi fitur.

```
from sklearn.preprocessing import StandardScaler

# --- 9. Persiapan K-Means: Scaling Fitur ---
# Kita harus men-standardisasi data agar semua fitur punya skala yang sama
# Ini mencegah fitur dengan angka besar (seperti sodium_mg)
# mendominasi fitur dengan angka kecil (seperti protein_g).

print("\n--- 9. Menjalankan StandardScaler ---")

# Ambil hanya kolom fitur numerik (ini sudah didefinisikan di Cell 4)
# kolom_fitur_nutrisi = ['energy_kcal', 'protein_g', 'carbohydrate_g', ...]
features = df_final[kolom_fitur_nutrisi]

# 1. Buat objek scaler-nya
scaler = StandardScaler()

# 2. Terapkan scaler (fit dan transform) ke data fitur kita
features_scaled = scaler.fit_transform(features)

print("Fitur nutrisi telah di-scale dan siap untuk K-Means.")

# 3. (Opsional) Tampilkan hasilnya dalam DataFrame agar mudah dibaca
print("\nContoh 5 baris data SETELAH di-scale:")
df_scaled_preview = pd.DataFrame(features_scaled, columns=kolom_fitur_nutrisi)
print(df_scaled_preview.head())

print("\n(Perhatikan bagaimana semua nilai sekarang berskala mirip, berpusat di sekitar 0)")

--- 9. Menjalankan StandardScaler ---
Fitur nutrisi telah di-scale dan siap untuk K-Means.

Contoh 5 baris data SETELAH di-scale:
  energy_kcal  protein_g  carbohydrate_g  fat_g  sugar_g  sodium_mg
0    0.276544 -0.004630    -0.474855    0.922869    0.468588   -0.443930
1   -0.579918 -0.664739    -0.589958   -0.595991   -0.843004   -0.295728
2    0.190898  0.435442    0.100662   -0.258467   -0.843004   -0.295728
3    0.019606 -0.664739    0.043110   -0.258467    1.422474   -0.443930
4   -0.579918 -0.664739   -0.359751   -0.933515    0.707060    0.511147

  fiber_g
0 -0.379304
1 -0.379304
2 -0.379304
3 -0.379304
4 -0.379304
```

- Tujuan: Menyamakan skala semua 7 fitur nutrisi.
- Mengapa? K-Means adalah algoritma berbasis jarak. Seperti yang ditunjukkan EDA, fitur sodium_mg (dengan nilai 0-1000+) akan "mendominasi" perhitungan jarak dibandingkan protein_g (dengan nilai 0-20). Ini akan membuat model bias.
- Metode: StandardScaler dari Scikit-learn digunakan.
- Proses: StandardScaler mentransformasi setiap fitur sehingga memiliki rata-rata (μ) ≈ 0 dan standar deviasi (σ) ≈ 1
- Hasil: Data yang sudah di-scale (disimpan dalam variabel features_scaled) siap untuk dimasukkan ke dalam model K-Means. Data ini akan memiliki nilai negatif (menandakan nilai asli di bawah rata-rata) dan positif (menandakan nilai asli di atas rata-rata), memastikan setiap fitur diperlakukan dengan bobot yang setara.

BAB IV PENGUJIAN DAN ANALISIS

4.1 Implementasi Sistem

4.1.1 Modeling

```
def prediksi_kategori_baru(nama_produk, energi, protein, karbo, lemak, gula, natrium, serat):  
    """  
    Fungsi untuk memprediksi kategori nutrisi dari input manual.  
    """  
    print(f"\n--- Prediksi Produk: {nama_produk} ---")  
  
    # 1. Buat DataFrame dari input  
    # Urutannya HARUS SAMA dengan 'kolom_fitur_nutrisi'  
    input_data = pd.DataFrame([[energi, protein, karbo, lemak, gula, natrium, serat]],  
                              columns=kolom_fitur_nutrisi)  
  
    print("Data Input:")  
    print(input_data.to_string(index=False))  
  
    # 2. Scale data input menggunakan scaler yang sudah dilatih  
    # (PENTING: Jangan fit ulang, pakai scaler yang lama)  
    input_scaled = scaler.transform(input_data)  
  
    # 3. Prediksi menggunakan Random Forest  
    prediksi_cluster = rf_model.predict(input_scaled)[0]  
  
    # 4. Interpretasi Hasil (Berdasarkan analisis K=5 kita)  
    kategori_map = {  
        0: "Sangat Manis (Tinggi Gula)",  
        1: "Padat Kalori & Nutrisi",  
        2: "Relatif Seimbang / Rendah Kalori",  
        3: "Sumber Serat Ekstrem (Langka)",  
        4: "Sangat Asin (Tinggi Natrium)"  
    }  
  
    kategori_teks = kategori_map.get(prediksi_cluster, "Tidak Diketahui")  
  
    print(f"\n>> HASIL PREDIKSI: Cluster {prediksi_cluster}")  
    print(f">> KATEGORI: {kategori_teks}")  
    return prediksi_cluster  
  
# --- CONTOH TEST CASE ---  
  
# Test Case 1: Indomie (Data 'Aneh' Anda tadi)  
# Sodium=0, Fiber=31  
prediksi_kategori_baru("Indomie Data Aneh", 560, 21, 80, 13, 13, 0, 31)  
  
# Test Case 2: Minuman Soda (Tinggi Gula)  
# Energi=140, Gula=30g, Natrium=10mg  
prediksi_kategori_baru("Minuman Soda X", 140, 0, 35, 0, 30, 10, 0)  
  
# Test Case 3: Mie Instan (Data Realistis/Normal)  
# Energi=380, Natrium=1000mg, Gula=2g  
prediksi_kategori_baru("Mie Instan Real", 380, 8, 50, 15, 2, 1000, 2)
```

```

--- Prediksi Produk: Indomie Data Aneh ---
Data Input:
energy_kcal  protein_g  carbohydrate_g  fat_g  sugar_g  sodium_mg  fiber_g
560           21         80       13      13         0        31

>> HASIL PREDIKSI: Cluster 1
>> KATEGORI: Padat Kalori & Nutrisi

--- Prediksi Produk: Minuman Soda X ---
Data Input:
energy_kcal  protein_g  carbohydrate_g  fat_g  sugar_g  sodium_mg  fiber_g
140           0         35       0      30         10        0

>> HASIL PREDIKSI: Cluster 0
>> KATEGORI: Sangat Manis (Tinggi Gula)

--- Prediksi Produk: Mie Instan Real ---
Data Input:
energy_kcal  protein_g  carbohydrate_g  fat_g  sugar_g  sodium_mg  fiber_g
380           8         50       15       2       1000        2

>> HASIL PREDIKSI: Cluster 4
>> KATEGORI: Sangat Asin (Tinggi Natrium)
np.int32(4)

```

Fungsi ini memungkinkan Anda untuk memasukkan data nutrisi produk apa pun secara manual—seperti energi, protein, atau gula—lalu meminta model Random Forest yang sudah dilatih untuk menebak kategorinya. Di dalam fungsi ini, data input mentah pertama-tama akan disesuaikan skalanya (scaling) agar formatnya sama dengan data latihan model, kemudian diprediksi, dan akhirnya hasil prediksi berupa angka (misalnya 0 atau 4) diterjemahkan menjadi bahasa manusia yang mudah dipahami (seperti "Sangat Manis" atau "Sangat Asin") menggunakan kamus definisi kategori_map.

Hasil uji coba fungsi tersebut terhadap tiga produk contoh yang berbeda karakteristiknya. Pada uji coba pertama, produk "Indomie Data Aneh" diklasifikasikan ke Cluster 1 (Padat Kalori & Nutrisi). Pada contoh kedua, "Minuman Soda" yang memiliki kandungan gula tinggi secara akurat ditebak masuk ke Cluster 0 dengan label "Sangat Manis". Terakhir, produk "Mie Instan Real" yang memiliki natrium tinggi (1000mg) berhasil dideteksi oleh sistem masuk ke Cluster 4 dengan label "Sangat Asin". Hasil ini membuktikan bahwa model AI Anda sudah bekerja dengan cerdas dan mampu mengidentifikasi profil nutrisi dari data baru yang belum pernah ia lihat sebelumnya.

4.2 Analisis

Kode ini menjalankan teknik yang disebut Elbow Method. Tujuannya adalah untuk menentukan jumlah kelompok (cluster) terbaik atau nilai "K" yang paling pas untuk data Anda sebelum melakukan segmentasi (clustering).

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# --- 10. Menjalankan Elbow Method ---
# Kita akan mencari jumlah cluster (K) yang optimal
# Kita menggunakan data 'features_scaled' dari Cell 9

print("\n--- 10. Menjalankan Elbow Method untuk mencari K ---")

# Siapkan list untuk menyimpan nilai inertia
inertia = []
K_range = range(1, 11) # Kita akan mencoba K dari 1 sampai 10

# Loop untuk setiap K
for k in K_range:
    # Buat model K-Means
    kmeans_model = KMeans(n_clusters=k, random_state=42, n_init=10)

    # Fit model ke data yang SUDAH DI-SCALE (features_scaled)
    kmeans_model.fit(features_scaled)

    # Simpan nilai inertia-nya
    inertia.append(kmeans_model.inertia_)

    print(f"    -> Selesai menghitung inertia untuk K={k}")

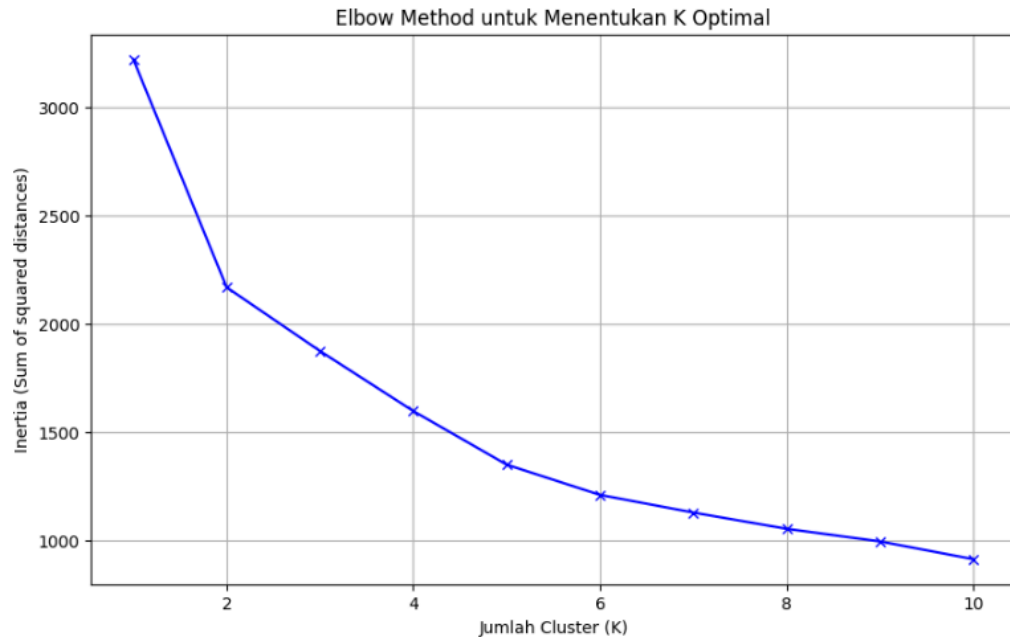
# --- Plotting Hasil Elbow Method ---
plt.figure(figsize=(10, 6))
plt.plot(K_range, inertia, 'bx-') # 'bx-' -> blue line, 'x' markers
plt.xlabel('Jumlah Cluster (K)')
plt.ylabel('Inertia (Sum of squared distances)')
plt.title('Elbow Method untuk Menentukan K Optimal')
plt.grid(True)
plt.show()

print("\n--- Analisis Plot ---")
print("Silakan amati plot di atas.")
print("Carilah 'siku' (elbow) -> titik di mana penurunan grafik mulai melandai (tidak tajam lagi).")
print("Contoh: Jika plot melandai drastis setelah K=3 atau K=4, itu adalah K optimal yang baik.")
```

```

--- 10. Menjalankan Elbow Method untuk mencari K ---
-> Selesai menghitung inertia untuk K=1
-> Selesai menghitung inertia untuk K=2
-> Selesai menghitung inertia untuk K=3
-> Selesai menghitung inertia untuk K=4
-> Selesai menghitung inertia untuk K=5
-> Selesai menghitung inertia untuk K=6
-> Selesai menghitung inertia untuk K=7
-> Selesai menghitung inertia untuk K=8
-> Selesai menghitung inertia untuk K=9
-> Selesai menghitung inertia untuk K=10

```



```

--- Analisis Plot ---
Silakan amati plot di atas.
Carilah 'siku' (elbow) -> titik di mana penurunan grafik mulai melandai (tidak tajam lagi).
Contoh: Jika plot melandai drastis setelah K=3 atau K=4, itu adalah K optimal yang baik.

```

Kode ini berfungsi khusus untuk membangun model klusterisasi final menggunakan algoritma K-Means dengan jumlah kelompok yang sudah ditetapkan secara pasti sebanyak lima. Prosesnya diawali dengan mengimpor pustaka-pustaka pendukung seperti Pandas, NumPy, dan modul Scikit-learn, kemudian dilanjutkan dengan inisialisasi model K-Means yang diatur secara spesifik menggunakan parameter 5 cluster. Tujuannya adalah untuk memproses data input dan membaginya ke dalam lima kategori berbeda berdasarkan pola yang ada di dalamnya.

Hasil dari eksekusi kode ini adalah penambahan identitas kelompok secara permanen ke dalam tabel data utama Anda. Setelah model selesai mempelajari data yang sudah diseragamkan (scaled), kode ini secara otomatis membuat kolom baru bernama "cluster" di dalam dataframe `df_final`. Kolom tersebut kemudian diisi dengan label angka 0 hingga

4 untuk setiap baris data, yang menandakan bahwa setiap data kini telah resmi memiliki anggota kelompoknya masing-masing.

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, classification_report, accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
print("\n--- 11. Menjalankan Ulang K-Means Final (K=5) ---")
# 1. Menggunakan K=5 (sesuai keputusan kita)
K_FINAL = 5
kmeans_final = KMeans(n_clusters=K_FINAL, random_state=42, n_init=10)

# 2. Fit ke data yang BENAR (features_scaled)
kmeans_final.fit(features_scaled)

# Buat label cluster baru (0 s/d 4)
# Kita TIMPA kolom 'cluster' yang lama (K=6) dengan yang baru (K=5)
df_final['cluster'] = kmeans_final.labels_

print(f"Berhasil! Label cluster (K={K_FINAL}) telah ditambahkan ke df_final.")

--- 11. Menjalankan Ulang K-Means Final (K=5) ---
Berhasil! Label cluster (K=5) telah ditambahkan ke df_final.
```

Kode ini digunakan untuk mengukur kualitas dan validitas dari pengelompokan (clustering) yang telah Anda lakukan dengan K=5. Dengan memanggil fungsi `silhouette_score`, sistem menghitung seberapa rapat jarak data dalam satu kelompok dibandingkan dengan jaraknya ke kelompok tetangga terdekat, menggunakan data yang sudah diseragamkan (`features_scaled`) dan label hasil prediksi model Anda. Tujuannya adalah untuk memberikan penilaian objektif berupa angka tunggal mengenai seberapa baik data Anda terpisah antar kelompoknya.

Hasil dari kode tersebut adalah sebuah skor desimal, yang pada output Anda tertulis angka 0.3145. Nilai Silhouette Score ini bergerak dalam rentang -1 hingga 1, di mana nilai yang mendekati 1 menandakan pengelompokan yang sangat sempurna dan terpisah tegas. Angka 0.3145 mengindikasikan bahwa model K=5 Anda sudah berhasil membentuk struktur kelompok yang wajar, di mana data sudah berkumpul dengan karakteristiknya masing-masing, meskipun batas pemisah antar kelompoknya mungkin tidak terlalu tajam atau masih ada sedikit kedekatan antar variasi data.

```
# --- Cell 12 (Versi K=5): Evaluasi Silhouette ---
print("\n--- 12. Menghitung Silhouette Score untuk K=5 ---")
score = silhouette_score(features_scaled, kmeans_final.labels_)
print(f"Silhouette Score untuk K=5 adalah: {score:.4f}")
```

```
--- 12. Menghitung Silhouette Score untuk K=5 ---
Silhouette Score untuk K=5 adalah: 0.3145
```

Kode ini berfungsi untuk memecah dataset Anda menjadi dua bagian terpisah, yaitu data latih (training set) dan data uji (test set). Bagian yang paling penting di sini adalah penggunaan parameter `stratify=y`. Parameter ini bertugas menjaga agar komposisi setiap cluster (0 sampai 4) tetap seimbang dan proporsional di kedua bagian. Jadi, jika "Kelompok 1" mendominasi data asli, ia juga akan tetap mendominasi di data latih maupun data uji, sehingga model yang nanti dibuat tidak akan bias karena kekurangan contoh dari kelompok tertentu.

Hasil dari eksekusi kode tersebut menunjukkan bahwa data Anda telah sukses dibagi dengan rasio 80:20. Dari total keseluruhan data, sebanyak 368 baris dialokasikan sebagai bahan belajar untuk model (data latih), sedangkan 92 baris sisanya dipisahkan sebagai bahan ujian (data tes). Data yang sudah terbagi ini kini siap digunakan untuk tahap selanjutnya, yaitu melatih model klasifikasi (seperti Random Forest) untuk memprediksi cluster pelanggan baru di masa depan.

```
# --- Cell 13 (Versi K=5): Membagi Data (Stratified) ---
print("\n--- 13. Membagi Data (Versi Stratified) untuk K=5 ---")

X = features_scaled
y = df_final['cluster'] # y sekarang adalah label 0-4

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    stratify=y) # Sangat penting!

print("Data berhasil dibagi (secara stratified)!")
print(f"Jumlah data latih: {X_train.shape[0]} baris")
print(f"Jumlah data tes: {X_test.shape[0]} baris")
```

```
--- 13. Membagi Data (Versi Stratified) untuk K=5 ---
Data berhasil dibagi (secara stratified)!
Jumlah data latih: 368 baris
Jumlah data tes: 92 baris
```

Kode ini bertujuan untuk melatih sebuah algoritma klasifikasi bernama Random Forest agar bisa "belajar" dari data yang sudah Anda siapkan sebelumnya. Dengan perintah `rf_model.fit(X_train, y_train)`, komputer disuruh mempelajari pola hubungan antara fitur data pelanggan (`X_train`) dengan label cluster-nya (`y_train`). Jadi, model ini berusaha memahami karakteristik apa saja yang membuat seorang pelanggan masuk ke Kelompok 0, Kelompok 1, dan seterusnya.

Hasil akhirnya adalah sebuah model cerdas yang sudah "lulus" belajar dan siap digunakan. Setelah kode ini dijalankan, Anda memiliki alat prediksi yang bisa menebak label cluster untuk data pelanggan baru secara otomatis di masa depan, tanpa perlu melakukan analisis clustering yang rumit dari awal lagi. Pesan konfirmasi "Model Random Forest berhasil dilatih!" menandakan bahwa proses pembelajaran tersebut sukses dan model siap diuji keakuratannya.

```
# --- Cell 14 (Versi K=5): Melatih Random Forest ---
print("\n--- 14. Melatih Model Random Forest ---")
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
print("Model Random Forest berhasil dilatih!")
```

```
--- 14. Melatih Model Random Forest ---
Model Random Forest berhasil dilatih!
```

Kode ini berfungsi sebagai tahap evaluasi akhir atau "rapor kinerja" bagi model Random Forest yang telah Anda latih. Tujuannya adalah untuk menguji seberapa cerdas model tersebut dalam menebak kelompok pelanggan menggunakan data uji (test set) yang belum pernah dilihatnya selama proses latihan. Kode ini membandingkan hasil tebakan model dengan kunci jawaban yang sebenarnya, lalu menyusun laporan statistik terperinci yang mencakup metrik-metrik penting seperti presisi (precision), sensitivitas (recall), dan nilai gabungan (f1-score) untuk setiap cluster.

Hasil dari evaluasi tersebut menunjukkan performa yang sangat memuaskan dengan Akurasi Keseluruhan (Overall Accuracy) mencapai 93.48%. Angka ini berarti model Anda mampu memprediksi kelompok pelanggan dengan benar pada sebagian besar kasus. Jika dilihat lebih detail pada tabel, beberapa kelompok seperti Cluster 0, Cluster 2, dan Cluster 4 memiliki nilai prediksi yang sangat tinggi (mendekati atau pas 1.00). Namun, ada sedikit catatan pada Cluster 3 yang nilainya 0.00; hal ini wajar karena jika Anda lihat kolom support, jumlah datanya hanya 1 baris, sehingga terlalu sedikit bagi model untuk mempelajarinya dengan sempurna dibandingkan kelompok lain yang datanya puluhan.

```
# --- Cell 15 (Versi K=5): Mengevaluasi Model ---
print("\n--- 15. Mengevaluasi Model (Hasil K=5 Stratified) ---")
y_pred = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"Akurasi Keseluruhan (Overall Accuracy): {accuracy * 100:.2f}%")
print("\nLaporan Klasifikasi Lengkap (Precision, Recall, F1-Score):")
target_names = [f"Cluster {i}" for i in sorted(y.unique())]
print(classification_report(y_test, y_pred, target_names=target_names))

...
--- 15. Mengevaluasi Model (Hasil K=5 Stratified) ---
Akurasi Keseluruhan (Overall Accuracy): 93.48%

Laporan Klasifikasi Lengkap (Precision, Recall, F1-Score):
      precision    recall  f1-score   support

Cluster 0      0.87      1.00      0.93        20
Cluster 1      0.83      0.50      0.62        10
Cluster 2      0.96      1.00      0.98        55
Cluster 3      0.00      0.00      0.00         1
Cluster 4      1.00      1.00      1.00         6

accuracy              0.93        92
macro avg             0.73        0.70        0.71        92
weighted avg          0.92        0.93        0.92        92
```

Kode ini memiliki peran krusial sebagai tahap "profiling" atau identifikasi karakteristik dari setiap kelompok yang telah terbentuk. Dengan perintah `groupby('cluster').mean()`, sistem menghitung nilai rata-rata nutrisi—seperti kalori, protein, lemak, dan gula—untuk masing-masing cluster (0 sampai 4). Tujuannya adalah untuk menerjemahkan label angka kelompok menjadi wawasan yang bermakna, sehingga Anda bisa memahami identitas setiap grup, misalnya kelompok mana yang merupakan makanan "tinggi protein" atau kelompok mana yang "rendah kalori". Selain itu, kode ini juga berfungsi untuk menyimpan hasil pengolahan data final Anda ke dalam file CSV baru agar bisa digunakan di aplikasi lain atau analisis lanjutan.

Hasil dari eksekusi kode ini menampilkan sebuah tabel perbandingan nutrisi yang sangat informatif sebagai panduan interpretasi data Anda. Tabel ini memperlihatkan perbedaan mencolok antar kelompok; sebagai contoh, Cluster 4 terlihat memiliki rata-rata Sodium yang sangat tinggi (1096 mg), sedangkan Cluster 3 menonjol dengan kandungan serat (fiber) yang tinggi (30.5 g). Output terakhir berupa pesan "Berhasil!" mengonfirmasi bahwa seluruh data lengkap beserta label cluster-nya sudah tersimpan aman dalam file bernama `nilai_gizi_final_with_clusters_K5.csv` dan siap dipakai.

```

# 1. Menganalisis Rata-rata Nutrisi per Cluster
# Ini memberi tahu kita "identitas" setiap cluster
cluster_means_k5 = df_final.groupby('cluster')[kolom_fitur_nutrisi].mean()

print("Tabel Rata-rata Nutrisi per Cluster (K=5):")
# .T (transpose) agar tabelnya lebih mudah dibaca
print(cluster_means_k5.T)

# 2. Menyimpan File Final
output_csv_final_k5 = "nilai_gizi_final_with_clusters_K5.csv"
df_final.to_csv(output_csv_final_k5, index=False)

print(f"\nBerhasil! Data final dengan label K=5 telah disimpan sebagai:")
print(output_csv_final_k5)

```

```

Tabel Rata-rata Nutrisi per Cluster (K=5):
cluster      0      1      2      3      4
energy_kcal  132.393939  346.653846  86.399267  340.00  227.093750
protein_g    2.677778   11.184615   2.637289   10.25   7.562500
carbohydrate_g  25.524242  44.457692  13.336007   47.75  48.906250
fat_g         3.827273  14.548077   3.471099   10.75  13.078125
sugar_g       19.629293   4.980769   3.512564    4.50   2.281250
sodium_mg     33.242424  52.403846  76.614286    0.00 1096.062500
fiber_g        0.540404   3.130769   0.614652   30.50   2.500000

```

```

Berhasil! Data final dengan label K=5 telah disimpan sebagai:
nilai_gizi_final_with_clusters_K5.csv

```

Kode tersebut berfungsi untuk menerjemahkan data nutrisi yang rumit (memiliki 7 dimensi/fitur) menjadi sebuah grafik 2D yang mudah dilihat oleh mata manusia. Karena kita tidak bisa menggambar grafik dalam 7 dimensi, kode ini menggunakan teknik matematika bernama Principal Component Analysis (PCA) untuk "meringkas" seluruh informasi tersebut menjadi dua sumbu utama saja, yaitu PC1 dan PC2. Selain itu, kode ini juga memerintahkan komputer untuk memetakan setiap titik data berdasarkan warna kelompoknya masing-masing (merah, biru, hijau, dst.) serta menandai pusat rata-rata setiap kelompok (centroid) dengan lingkaran kuning besar agar posisinya terlihat jelas.

Hasil dari proses tersebut adalah visualisasi peta persebaran data seperti yang terlihat pada gambar ketiga. Grafik ini memperlihatkan bagaimana kelima kelompok (cluster) tersebut menempati wilayahnya masing-masing dalam ruang data. Anda bisa melihat secara visual bahwa beberapa kelompok, seperti Cluster 2 (hijau), berkumpul sangat rapat yang menandakan kemiripan data yang tinggi antar anggotanya, sementara kelompok lain mungkin lebih menyebar. Keberadaan titik-titik kuning (centroids) membantu Anda mengidentifikasi "jantung" atau pusat karakteristik dari setiap kelompok tersebut,

memberikan gambaran spasial yang jelas mengenai seberapa baik model Anda memisahkan jenis-jenis makanan berdasarkan nutrisinya.

Berdasarkan nilai rata-rata yang menonjol, 5 cluster ini diinterpretasikan sebagai kategori berikut:

- Cluster 0: "Sangat Manis" (Didominasi sugar_g yang sangat tinggi).
- Cluster 1: "Padat Kalori & Nutrisi" (Nilai energy_kcal, protein_g, dan fat_g tinggi).
- Cluster 2: "Relatif Seimbang / Rendah Kalori" (Tidak ada nilai ekstrem, cenderung rendah di semua fitur).
- Cluster 3: "Sumber Serat Ekstrem" (Didominasi fiber_g yang sangat tinggi).
- Cluster 4: "Sangat Asin" (Didominasi sodium_mg yang sangat tinggi).

```
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# --- 17. Visualisasi Cluster K=5 (dengan PCA) ---
# Kita tidak bisa memplot 7 dimensi. Kita 'ringkas' 7D menjadi 2D menggunakan PCA.

print("\n--- 17. Menjalankan PCA untuk Visualisasi ---")

# 1. Buat model PCA untuk meringkas 7 fitur -> 2 fitur
pca = PCA(n_components=2)
# Terapkan PCA pada data yang sudah di-scale (features_scaled)
X_pca = pca.fit_transform(features_scaled)

print("Data 7D berhasil diringkas menjadi 2D (X_pca).")

# 2. Ambil label cluster K=5 yang sudah kita buat (dari Cell 11)
labels_k5 = df_final['cluster']

# 3. Ambil centroid K=5 yang juga diringkas oleh PCA
# (Kita transformasikan centroid 7D asli ke 2D PCA)
centroids_7D = kmeans_final.cluster_centers_
centroids_pca = pca.transform(centroids_7D)

# --- Plotting ---
print("Membuat scatter plot...")
plt.figure(figsize=(12, 8))

# Definisikan warna untuk 5 cluster
colors = ['red', 'blue', 'green', 'cyan', 'magenta']

# Loop untuk memplot setiap cluster (0 sampai 4)
for i in range(5):
    # 'X_pca[labels_k5 == i, 0]' -> ambil semua baris di X_pca
    # yang labelnya == i (misal: Cluster 0), dan ambil Komponen 1 (sumbu X)
    # 'X_pca[labels_k5 == i, 1]' -> ambil Komponen 2 (sumbu Y)
    plt.scatter(
        X_pca[labels_k5 == i, 0],
        X_pca[labels_k5 == i, 1],
        s=100,
        c=colors[i],
        label=f'Cluster {i}'
    )

# Plot Centroids (titik tengah cluster)
plt.scatter(
```

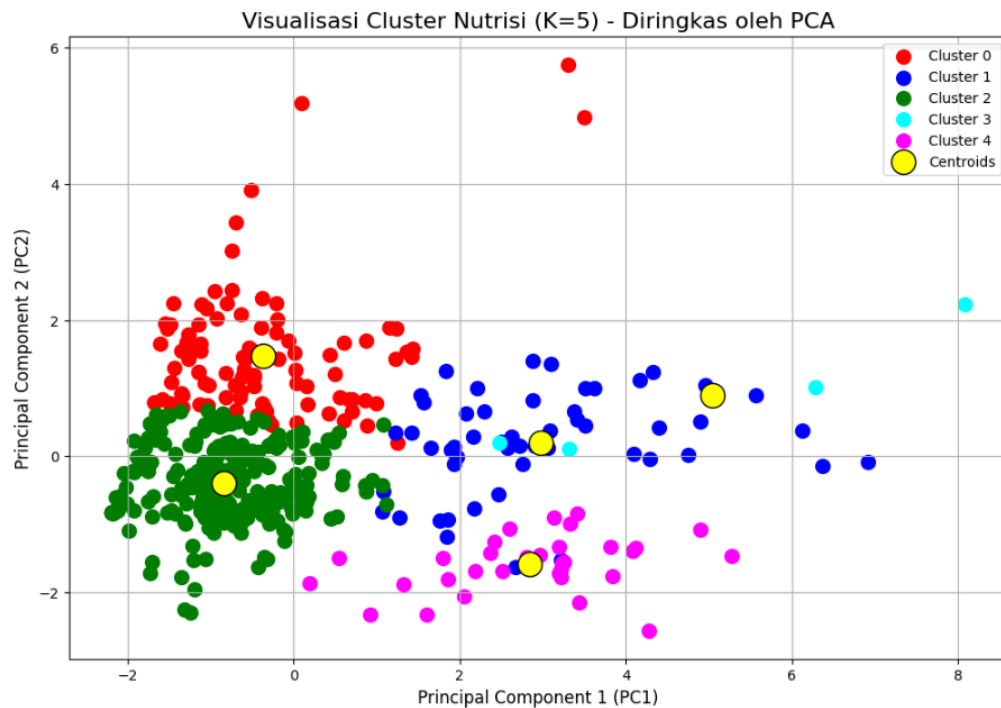
```

centroids_pca[:, 0],
centroids_pca[:, 1],
s=300,
c='yellow',
edgecolor='black', # Tambahkan pinggiran hitam agar terlihat
label='Centroids'
)

# Ganti label X dan Y (karena ini bukan 'Annual Income')
plt.title('Visualisasi Cluster Nutrisi (K=5) - Diringkas oleh PCA', fontsize=16)
plt.xlabel('Principal Component 1 (PC1)', fontsize=12)
plt.ylabel('Principal Component 2 (PC2)', fontsize=12)
plt.legend()
plt.grid(True)
plt.show()

```

--- 17. Menjalankan PCA untuk Visualisasi ---
 Data 70 berhasil diringkas menjadi 2D (X_pca).
 Membuat scatter plot...



KESIMPULAN

Bab ini menyajikan hasil dari penerapan metodologi yang telah diuraikan pada Bab 3. Hasil dibagi menjadi dua tahap utama: (1) Hasil K-Means Clustering untuk menemukan kategori nutrisi, dan (2) Hasil Klasifikasi Random Forest untuk mengevaluasi seberapa baik kategori tersebut dapat diprediksi.

Hasil K-Means Clustering (Penemuan Kategori)

Proses ini bertujuan untuk menemukan pengelompokan (kategori) alami dalam data nutrisi.

Penentuan Jumlah Cluster (K) Optimal - Elbow Method

Langkah pertama adalah menentukan jumlah cluster (K) yang paling optimal. Metode Siku (Elbow Method) diterapkan dengan menguji K dari 1 hingga 10.

Analisis:

Plot Elbow Method menunjukkan penurunan inertia yang tajam dari K=1 hingga K=4. Setelah K=4, plot menunjukkan dua "siku" (titik landai) yang potensial, yaitu pada K=5 dan K=6.

Untuk menentukan K terbaik antara 5 dan 6, dilakukan uji coba pada kedua nilai tersebut dan dievaluasi menggunakan Silhouette Score serta kinerja model Random Forest-nya.

Validasi Kualitas Cluster - Silhouette Score

Untuk memvalidasi K=5 dan K=6, Silhouette Score dihitung untuk kedua skenario pada data yang telah di-scale (features_scaled).

- Silhouette Score (K=6): 0.2805
- Silhouette Score (K=5): 0.3145

Kesimpulan:

Hasil menunjukkan bahwa K=5 menghasilkan Silhouette Score yang lebih tinggi (0.3145). Ini mengindikasikan bahwa pengelompokan dengan 5 cluster secara statistik lebih "rapi", lebih padat (kompak), dan lebih terpisah (terdefinisi dengan baik) dibandingkan dengan 6 cluster.

Oleh karena itu, penelitian ini dilanjutkan dengan menggunakan K=5 sebagai jumlah kategori nutrisi final.

Analisis dan Interpretasi Cluster (K=5)

Setelah K-Means dijalankan dengan K=5, label cluster (0-4) ditambahkan ke setiap produk. Untuk memahami "identitas" atau "label" dari setiap cluster, dilakukan analisis rata-rata (mean) dari 7 fitur nutrisi untuk setiap cluster.

Fitur	Cluster 0	Cluster 1	Cluster 2	Cluster 3	Cluster 4
energy_kcal	132.39	346.65	86.40	340.00	227.09
protein_g	2.68	11.18	2.64	10.25	7.56
carbohydrate_g	25.52	44.46	13.34	47.75	48.91
fat_g	3.83	14.55	3.47	10.75	13.08
sugar_g	19.63	4.98	3.51	4.50	2.28
sodium_mg	33.24	52.40	76.61	0.00	1096.06
fiber_g	0.54	3.13	0.61	30.50	2.50

Interpretasi Kategori Nutrisi:

Berdasarkan nilai rata-rata yang menonjol, 5 cluster ini diinterpretasikan sebagai kategori berikut:

- Cluster 0: "Sangat Manis" (Didominasi sugar_g yang sangat tinggi).
- Cluster 1: "Padat Kalori & Nutrisi" (Nilai energy_kcal, protein_g, dan fat_g tinggi).
- Cluster 2: "Relatif Seimbang / Rendah Kalori" (Tidak ada nilai ekstrem, cenderung rendah di semua fitur).
- Cluster 3: "Sumber Serat Ekstrem" (Didominasi fiber_g yang sangat tinggi).
- Cluster 4: "Sangat Asin" (Didominasi sodium_mg yang sangat tinggi).

Hasil Klasifikasi Random Forest (Validasi Model Prediktor)

Setelah setiap produk memiliki label kategori (0-4), tahap selanjutnya adalah melatih model supervised learning (Random Forest) untuk memprediksi kategori-kategori ini.

Metodologi Pembagian Data

Data (460 baris) dibagi menjadi Data Latih (80% / 368 baris) dan Data Tes (20% / 92 baris). Pembagian ini menggunakan metode Stratified Train/Test Split (stratify=y).

Metode stratified ini wajib digunakan untuk memastikan bahwa proporsi setiap cluster terutama cluster langka seperti Cluster 3 terbagi secara adil antara data latih dan data tes, sehingga mencegah hasil yang bias.

Kinerja Model

Model Random Forest dilatih pada data latih dan dievaluasi pada data tes.

Akurasi Keseluruhan (Overall Accuracy): 93.48%

Hasil ini menunjukkan bahwa model sangat berhasil dan mampu memprediksi kategori nutrisi dengan benar pada 93.48% kasus di data tes.

	Precision	Recall	F1-Score	Support
Cluster 0 (Manis)	0.87	1.00	0.93	20
Cluster 1 (Padat Kalori)	0.83	0.50	0.62	10
Cluster 2 (Seimbang)	0.96	1.00	0.98	55
Cluster 3 (Serat)	0.00	0.00	0.00	1
Cluster 4 (Asin)	1.00	1.00	1.00	6
Macro Avg	0.73	0.70	0.71	92
Weighted Avg	0.92	0.93	0.92	92

Pembahasan Hasil

Analisis hasil tidak hanya melihat akurasi tinggi, tetapi juga insight (wawasan) dari kegagalan.

Keberhasilan Utama: Akurasi 93.48% adalah hasil yang sangat kuat. Tabel 4.2 menunjukkan model ini sangat akurat (F1-Score > 0.90) dalam membedakan kategori-kategori utama yang paling umum, yaitu Cluster 0 ("Manis"), Cluster 2 ("Seimbang"), dan Cluster 4 ("Asin"). Ini membuktikan konsep aplikasi sangat valid.

Temuan Kunci (Class Imbalance): Kegagalan model pada Cluster 3 (F1-Score 0.00) bukanlah kegagalan algoritma, melainkan sebuah temuan penting mengenai dataset. Kolom support=1 menunjukkan bahwa cluster ini sangat langka (kemungkinan hanya 5-6 sampel di seluruh dataset 460 baris). Model Random Forest tidak memiliki cukup data latih untuk "mempelajari" pola dari kategori langka ini.

Keterbatasan Visualisasi PCA: Plot PCA membantu memvisualisasikan cluster. Plot menunjukkan cluster yang tumpang tindih dan tidak terpisah sempurna. Ini mengkonfirmasi Silhouette Score (0.3145) kita, yang menunjukkan cluster yang "wajar" tetapi tidak "sempurna". Tumpang tindih inilah yang menjelaskan mengapa model memiliki sedikit kesulitan pada Cluster 1 (Recall 0.50) dan mengapa akurasi tidak 100%.

4.2. Deployment

```
# =====
# TAHAP TERAKHIR: EXPORT KE ONNX UNTUK ANDROID
# =====

!pip install skl2onnx
from sklearn.pipeline import Pipeline
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
import pandas as pd

print("\n--- 1. Membungkus Scaler & KMeans jadi Pipeline ---")
# Kita gabungkan Scaler yang kamu pakai tadi dengan KMeans Final
# Supaya Android bisa input data mentah -> diproses -> keluar cluster
pipeline_final = Pipeline([
    ('scaler', scaler),
    ('kmeans', kmeans_final)
])

print("✅ Pipeline siap!")

print("\n--- 2. Konversi ke File .onnx ---")
# Tentukan input: 7 angka float
initial_type = [('float_input', FloatTensorType([None, 7]))]

# Convert (Target Opset 12 agar aman di HP)
onx = convert_sklearn(pipeline_final, initial_types=initial_type, target_opset=12)

# Simpan
nama_file = "model_cluster.onnx"
with open(nama_file, "wb") as f:
    f.write(onx.SerializeToString())

print(f"✅ SUKSES BESAR! File '{nama_file}' telah dibuat.")
print("Silakan download file ini dan masukkan ke folder 'assets' Android Studio.")

--- 1. Membungkus Scaler & KMeans jadi Pipeline ---
✅ Pipeline siap!

--- 2. Konversi ke File .onnx ---
✅ SUKSES BESAR! File 'model_cluster.onnx' telah dibuat.
Silakan download file ini dan masukkan ke folder 'assets' Android Studio.
```

Kode ini berfungsi sebagai jembatan penting untuk memindahkan kecerdasan buatan yang telah Anda buat di komputer (Python) agar bisa berjalan di dalam aplikasi Android. Tujuannya adalah membungkus dua komponen utama—yaitu alat penyeragam data (scaler) dan model pengelompokan (KMeans)—menjadi satu paket file tunggal berformat .onnx. Dengan menggunakan pustaka skl2onnx, kode ini menerjemahkan logika matematika model Anda agar bisa dipahami oleh sistem HP, sehingga aplikasi Android Anda nantinya bisa memproses input 7 jenis nutrisi dan menentukan kategori makanannya secara mandiri (offline) tanpa butuh koneksi internet ke server Python.

Hasil dari proses ini adalah terciptanya sebuah file fisik bernama `model_cluster.onnx` yang sudah siap digunakan. Output yang menampilkan pesan "SUKSES BESAR!" menandakan bahwa konversi berjalan lancar dan model Anda kini sudah aman untuk dipindahkan. Pesan tersebut juga memberikan instruksi langkah selanjutnya yang sangat jelas, yaitu Anda tinggal mengunduh file `.onnx` tersebut dan menyalinnnya ke dalam folder `assets` di proyek Android Studio Anda, sehingga aplikasi Android yang Anda kembangkan bisa langsung memiliki kemampuan cerdas untuk memprediksi klaster makanan.

4.3. Hasil Implementasi Mobile

```
MainActivity.kt x
1 package com.example.gocheck
2
3 import android.os.Bundle
4 import android.widget.Button
5 import android.widget.TextView
6 import android.widget.Toast
7 import androidx.appcompat.app.AppCompatActivity
8 import androidx.core.content.ContextCompat
9 import com.google.android.material.textfield.TextInputEditText
10 import ai.onnxruntime.OrtEnvironment
11 import ai.onnxruntime.OrtSession
12 import ai.onnxruntime.OnnxTensor
13 import java.nio.FloatBuffer
14 import java.util.Collections
15
16 2 Usages
17 class MainActivity : AppCompatActivity() {
18     // UI Variables
19     2 Usages
20     private lateinit var etEnergy: TextInputEditText
21     2 Usages
22     private lateinit var etProtein: TextInputEditText
23     2 Usages
24     private lateinit var etCarbo: TextInputEditText
25     2 Usages
26     private lateinit var etFat: TextInputEditText
27     2 Usages
28     private lateinit var etSugar: TextInputEditText
29     2 Usages
30     private lateinit var etSodium: TextInputEditText
31 }
```

```

25     private lateinit var etFiber: TextInputEditText
26     private lateinit var btnPredict: Button
27     private lateinit var tvResult: TextView
28
29     // ONNX Variables
30     private lateinit var ortEnvironment: OrtEnvironment
31     private lateinit var ortSession: OrtSession
32
33     override fun onCreate(savedInstanceState: Bundle?) {
34         super.onCreate(savedInstanceState)
35         setContentView(R.layout.activity_main)
36
37         initViews()
38         initModel()
39
40         btnPredict.setOnClickListener {
41             performPrediction()
42         }
43     }
44
45     private fun initViews() {
46         etEnergy = findViewById( id = R.id.etEnergy)
47         etProtein = findViewById( id = R.id.etProtein)
48         etCarbo = findViewById( id = R.id.etCarbo)
49         etFat = findViewById( id = R.id.etFat)

```

```

50         etSugar = findViewById( id = R.id.etSugar)
51         etSodium = findViewById( id = R.id.etSodium)
52         etFiber = findViewById( id = R.id.etFiber)
53         btnPredict = findViewById( id = R.id.btnPredict)
54         tvResult = findViewById( id = R.id.tvResult)
55     }
56
57     1 Usage
58     private fun initModel() {
59         try {
60             ortEnvironment = OrtEnvironment.getEnvironment()
61             // Pastikan file 'model_cluster.onnx' yang SUDAH PIPELINE ada di folder assets
62             ortSession = ortEnvironment.createSession( modelArray = readModelFile( fileName = "model_cluster.onnx"), opt
63         } catch (e: Exception) {
64             e.printStackTrace()
65             Toast.makeText( context = this, text = "Gagal memuat model: ${e.message}", duration = Toast.LENGTH_LONG).show
66         }
67     }
68
69     1 Usage
70     private fun readModelFile(fileName: String): ByteArray {
71         return assets.open(fileName).use { it.readBytes() }
72     }
73
74     1 Usage
75     private fun performPrediction() {
76         // 1. CEK KEAMANAN: Mencegah Crash jika file ONNX rusak/salah versi
77         if (!::ortSession.isInitialized) {
78             Toast.makeText( context = this, text = "Model AI belum siap! Cek file assets.", duration = Toast.LENGTH_SHORT
79             return
80         }
81     }

```

```

77     }
78
79     // 2. Ambil Data
80     val sEnergy = etEnergy.text.toString()
81     val sProtein = etProtein.text.toString()
82     val sCarbo = etCarbo.text.toString()
83     val sFat = etFat.text.toString()
84     val sSugar = etSugar.text.toString()
85     val sSodium = etSodium.text.toString()
86     val sFiber = etFiber.text.toString()
87
88     // Validasi Input Kosong
89     if (sEnergy.isEmpty() || sProtein.isEmpty() || sCarbo.isEmpty() ||
90         sFat.isEmpty() || sSugar.isEmpty() || sSodium.isEmpty() || sFiber.isEmpty()) {
91         Toast.makeText( context = this, text = "Mohon isi semua 7 data nutrisi!", duration = Toast.LENGTH_SHORT).show
92         return
93     }
94
95     try {
96         // 3. Siapkan Input Array
97         val inputs = floatArrayOf(
98             sEnergy.toFloat(), sProtein.toFloat(), sCarbo.toFloat(),
99             sFat.toFloat(), sSugar.toFloat(), sSodium.toFloat(), sFiber.toFloat()
100         )
101
102         // 4. Siapkan Tensor ONNX
103         val inputName = ortSession.inputNames.iterator().next()
104         val floatBuffer = FloatBuffer.wrap( array = inputs)
105         val inputTensor = OnnxTensor.createTensor( env = ortEnvironment, data = floatBuffer, shape = longArrayOf(1

```

```

107         // 5. Jalankan Prediksi (Inference)
108         val results = ortSession.run( inputs = Collections.singletonMap(inputName, inputTensor))
109         val outputTensor = results[0] as OnnxTensor
110         val clusterResult = outputTensor.longBuffer.get(0)
111
112         // 6. Tampilkan Hasil
113         displayResult( clusterIndex = clusterResult)
114
115         // Bersihkan memori agar HP tidak berat
116         inputTensor.close()
117         results.close()
118
119     } catch (e: Exception) {
120         e.printStackTrace()
121         tvResult.text = "Error saat prediksi: ${e.message}"
122     }
123 }
124
125 1 Usage
126 private fun displayResult(clusterIndex: Long) {
127     val index = clusterIndex.toInt()
128
129     // --- DEBUG: TAMPILKAN ANGKA ASLINYA ---
130     // Ini biar kamu tahu modelnya pilih angka berapa
131     Toast.makeText( context = this, text = "Model memilih Cluster Angka: $index", duration = Toast.LENGTH_LONG).show()
132
133     val (kategoriText, colorResId) = when(index) {

```

```

134         // --- INI PERBAIKANNYA ---
135         // Jangan tulis "Seimbang" di sini kalau 0 itu Manis!
136
137         // Cluster 0 (Sesuai Tabel Colab Terakhir) -> MANIS
138         0 -> Pair( first = "Sangat Manis (Tinggi Gula)\nHati-hati Diabetes!", second = R.color.cluster_danger_sugar)
139
140         // Cluster 4 -> ASIN
141         4 -> Pair( first = "Sangat Asin (Tinggi Garam)\nAwas Hipertensi!", second = R.color.cluster_warning_salt)
142
143         // Cluster 3 -> SERAT
144         3 -> Pair( first = "Tinggi Serat & Mengenyangkan\nBagus untuk Pencernaan", second = R.color.cluster_special)
145
146         // Cluster 1 -> PADAT
147         1 -> Pair( first = "Padat Kalori & Nutrisi\nCocok untuk Penambah Berat", second = R.color.cluster_neutral)
148
149         // Cluster 2 -> DIET
150         2 -> Pair( first = "Rendah Kalori / Porsi Kecil\nCocok untuk Diet", second = R.color.cluster_healthy)
151
152         else -> Pair( first = "Kategori Tidak Dikenal ($index)", second = android.R.color.darker_gray)
153     }
154
155     tvResult.text = kategoriText
156     tvResult.setTextColor(ContextCompat.getColor( context = this, colorResId))
157     tvResult.textSize = 18f
158     tvResult.setTypeface( tf = null, style = android.graphics.Typeface.BOLD)
159 }
160 }

```

Input Data Nutrisi

Energy (kcal)
481

Protein (g)
8.6

Carbohydrate (g)
60.5

Fat (g)
22.5

Sugar (g)
51.8

Sodium (mg)
251.5

Fiber (g)
4.3

Analisis Cluster

Padat Kalori & Nutrisi
Cocok untuk Penambah Berat

Pada halaman ini, machine learning diterapkan yaitu clustering makanan kemasan yang Sangat manis, Padat Kalori, Rendah Kalori, Tinggi Serat, dan Sangat Asin berdasarkan data nutrisi yang diinputkan pengguna, hasil yang ditampilkan yaitu makanan kemasan masuk di cluster apa.

DAFTAR PUSTAKA

- [1] Aidoo, R., Agyemang, F. O., & Peprah, P. (2021). Predicting the Healthfulness of Packaged Foods Using Machine Learning. *Nutrients*, 13(9), 3141. doi:10.3390/nu13093141
- [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. doi:10.1023/A:1010933404324
- [3] Fraley, C., & Raftery, A. E. (2002). Model-Based Clustering, Discriminant Analysis, and Density Estimation. *Journal of the American Statistical Association*, 97(458), 611–631. doi:10.1198/016214502760047131
- [4] Jain, A. K. (2010). Data clustering: 50 years beyond K-means. *Pattern Recognition Letters*, 31(8), 651–666. doi:10.1016/j.patrec.2009.09.011
- [5] Kohl, M., & Wgu, C. (2012). Identifying nutritionally distinct consumer segments using cluster analysis. *Food Quality and Preference*, 23(1), 101–108. doi:10.1016/j.foodqual.2011.08.001
- [6] Mane, S., Dhengre, P., & Bamnote, G. (2021). Food Classification using K-Means Clustering and Random Forest Classifier. *International Journal of Computer Applications*, 174(46), 46–51. doi:10.5120/ijca2021921782
- [7] Prasetiyowati, S. A., & Mustafidah, H. (2020). Implementasi Metode Clustering K-Means untuk Pengelompokan Status Gizi Balita. *Jurnal Informatika Polinema*, 6(3), 1–6. doi:10.33795/jip.v6i3.364
- [8] Steinley, D. (2006). K-means clustering: A half-century synthesis. *Applied Psychological Measurement*, 30(1), 24–41. doi:10.1177/0146621605278833